甘特图管理系统 — 软件文档

文档类型： 程序设计说明书、功能规格说明书、用户手册
版本： 1.0
日期： 2025年3月

目录

1. 第一部分 程序设计说明书
2. 第二部分 功能规格说明书
3. 第三部分 用户手册
4. 第四部分 系统流程图
5. 第五部分 开发情况
6. 第六部分 测试结果

第一部分 程序设计说明书

1.1 程序概述

1.1.1 项目背景

甘特图管理系统是一套基于 Web 的项目任务管理软件，用于帮助用户可视化地规划、跟踪和管理项目进度。系统采用甘特图作为核心展示形式，支持任务的创建、编辑、删除、依赖关系设置、版本历史记录等完整功能。

1.1.2 开发目标

本系统的开发目标包括：（1）提供直观的甘特图界面，支持任务的时间轴展示和进度可视化；（2）支持任务之间的依赖关系定义，便于项目管理；（3）提供版本快照功能，可回溯

历史状态；（4）支持用户注册、登录、多项目隔离；（5）提供任务生命周期审计，记录每次变更。

1.1.3 技术选型

系统采用以下技术栈：前端使用 Next.js 16 框架和 React 19，配合 Redux Toolkit 进行状态管理；后端采用 Next.js App Router 的 API Routes 实现 RESTful 接口；数据库使用 PostgreSQL 存储用户、项目、任务、依赖关系及版本数据；样式采用 Tailwind CSS 4；认证采用 JWT（JSON Web Token）和 bcrypt 密码加密。

1.1.4 运行环境

推荐运行环境：Node.js 20+，PostgreSQL 14+，现代浏览器（Chrome、Edge、Firefox、Safari 最新版）。开发环境需配置数据库连接（lib/db.ts）及环境变量。

1.2 系统架构

1.2.1 整体架构

系统采用典型的三层架构：表示层（Presentation Layer）、业务逻辑层（Business Logic Layer）、数据访问层（Data Access Layer）。表示层由 Next.js 的页面和 React 组件构成；业务逻辑层由 API Routes 和 Redux slices 实现；数据访问层通过 pg 库直接访问 PostgreSQL。

1.2.2 目录结构

项目根目录主要包含：app/（Next.js 页面与 API 路由）、components/（React 组件）、store/（Redux 状态）、lib/（工具与数据库）、types/（TypeScript 类型定义）、scripts/（迁移与种子脚本）、public/（静态资源）。

1.2.3 数据流

用户操作触发组件事件，组件通过 dispatch 更新 Redux 状态或调用 fetch 请求 API；API 校验 JWT、查询/更新数据库后返回 JSON；前端根据响应更新状态并重绘界面。任务和依赖的增删改均通过 API 持久化，并可选触发版本快照。

1.3 数据库设计

1.3.1 用户表（users）

字段包括：id（UUID 主键）、username（唯一）、email（唯一）、password_hash、created_at。用于存储注册用户信息，密码以 bcrypt 哈希存储。

1.3.2 项目表（projects）

字段包括：id、user_id（外键 users）、name、start_date、end_date、status_date、created_at。每个项目属于一个用户，status_date 用于计算任务完成度的时间基准。

1.3.3 任务表（tasks）

字段包括：id、project_id、parent_id（自引用）、task_code、name、assignee、start_date、end_date、duration、duration_unit、percent_done、is_milestone、note、order_index、is_deleted、deleted_at、created_at、updated_at。支持层级任务（父子关系）、软删除、任务编码（如 T-001）。

1.3.4 依赖表（dependencies）

字段包括：id、project_id、from_task_id、to_task_id、type、lag。表示任务间的先后依赖，type 为依赖类型（如 FS 完成-开始），lag 为滞后量。

1.3.5 任务生命周期表（task_lifecycle）

记录任务每次变更：event_type（created/updated/moved/deleted）、field_name、old_value、new_value、description、created_by、created_at。用于审计与历史查看。

1.3.6 版本与变更日志表

project_versions 存储版本快照（snapshot JSONB）；change_logs 存储每个版本的变更明细，便于版本对比与回溯。

1.4 模块设计

1.4.1 认证模块

负责用户注册、登录、登出。注册时校验用户名和邮箱唯一性，密码经 bcrypt 加密后存入 users 表；登录成功后签发 JWT，前端将 token 存入 cookie 或 localStorage；后续请求在

Authorization 头或 Cookie 中携带 token，中间件（lib/middleware.ts、lib/auth.ts）负责解析与校验。

1.4.2 项目模块

提供项目的增删改查。用户仅能访问自己创建的项目。API：GET/POST /api/projects，GET/PUT/DELETE /api/projects/[id]。项目列表在仪表盘展示，支持新建项目并跳转到甘特图页面。

1.4.3 任务模块

提供任务的 CRUD 及批量操作。API：GET/POST /api/tasks/[projectId]，GET/PUT/DELETE /api/tasks/[projectId]/[taskId]。支持创建子任务、编辑任务属性、软删除、拖拽调整时间与顺序。每次变更可写入 task_lifecycle。

1.4.4 依赖模块

API：GET/POST/DELETE /api/dependencies/[projectId]。支持创建、删除任务间依赖；甘特图界面支持从任务条拖线到另一任务条创建依赖。删除任务时级联删除相关依赖。

1.4.5 版本模块

API：GET /api/versions/[projectId] 获取版本列表；POST 创建新版本；GET /api/versions/[projectId]/[versionId] 获取某版本详情及变更日志。版本快照包含当前任务与依赖的完整 JSON，便于回溯与对比。

1.5 接口设计

1.5.1 认证接口

POST /api/auth/register：body 含 username、email、password；返回用户信息及 token。
POST /api/auth/login：body 含 username、password；返回用户信息及 token。
POST /api/auth/logout：清除服务端会话或 token 黑名单（若实现）。

1.5.2 项目接口

GET /api/projects：返回当前用户的项目列表。

POST /api/projects：body 含 name、start_date 等；创建新项目。
GET /api/projects/[id]：返回项目详情。
PUT /api/projects/[id]：更新项目。
DELETE /api/projects/[id]：删除项目（级联删除任务与依赖）。

1.5.3 任务接口

GET /api/tasks/[projectId]：返回项目的任务列表及依赖关系。
POST /api/tasks/[projectId]：body 含 name、parent_id、start_date 等；创建任务，自动生成 task_code。
GET /api/tasks/[projectId]/[taskId]：返回任务详情及 lifecycle 事件。
PUT /api/tasks/[projectId]：body 含任务数组；批量更新任务。
DELETE /api/tasks/[projectId]/[taskId]：软删除任务。

1.5.4 依赖接口

GET /api/dependencies/[projectId]：返回项目依赖列表（也可由任务接口一并返回）。
POST /api/dependencies/[projectId]：body 含 from_task_id、to_task_id、type、lag；创建依赖。
DELETE /api/dependencies/[projectId]：body 含 id；删除依赖。

1.5.5 版本接口

GET /api/versions/[projectId]：返回版本列表（含版本号、创建时间、变更数量）。
POST /api/versions/[projectId]：基于当前任务状态创建新版本。
GET /api/versions/[projectId]/[versionId]：返回版本快照及 change_logs。

第二部分 功能规格说明书

2.1 功能概述

系统主要功能包括：用户管理、项目管理、任务管理、甘特图展示与交互、依赖管理、版本历史、任务生命周期审计。

2.2 用户管理功能

2.2.1 用户注册

用户可填写用户名、邮箱、密码进行注册。系统校验：用户名和邮箱在数据库中唯一；密码长度与复杂度可配置。注册成功后自动登录，跳转到仪表盘。

2.2.2 用户登录

用户输入用户名和密码登录。系统校验密码哈希，通过则签发 JWT，有效期内可访问需认证的接口。登录失败返回错误提示。

2.2.3 用户登出

用户点击登出后，清除前端 token 并跳转到登录页。服务端可选择性使 token 失效。

2.3 项目管理功能

2.3.1 项目列表

仪表盘展示当前用户的所有项目，以卡片形式显示项目名称、起止日期等。支持点击进入项目甘特图。

2.3.2 创建项目

用户可输入项目名称创建新项目。系统自动设置 start_date 为当前日期，并可配置默认模板。

2.3.3 编辑与删除项目

用户可修改项目名称、起止日期、status_date 等；删除项目将级联删除其下所有任务与依赖，操作需确认。

2.4 任务管理功能

2.4.1 创建任务

在甘特图中可创建根任务或子任务。必填项为任务名称；可选项包括开始日期、结束日期、工期、责任人、完成度、是否里程碑、备注等。系统自动生成 task_code（如 T-001、T-002）。

2.4.2 编辑任务

通过双击任务或点击编辑按钮打开编辑弹窗。可修改名称、日期、责任人、完成度、里程碑、备注等。保存后更新数据库并可选记录 lifecycle。

2.4.3 删除任务

支持软删除：将 is_deleted 置为 true，deleted_at 记录时间。甘特图不展示已删除任务，依赖关系相应过滤。

2.4.4 任务层级

支持父子任务层级结构。父任务可展开/折叠子任务；子任务可缩进显示。parent_id 指向父任务 id，根任务 parent_id 为 null。

2.4.5 任务拖拽

支持拖拽任务条调整开始/结束时间（resize）；支持拖拽任务行调整顺序（reorder）；拖拽时考虑依赖约束，避免产生循环依赖。

2.5 甘特图展示功能

2.5.1 时间轴

顶部显示日期刻度，支持按天、周、月缩放。列宽可调（colW），影响时间粒度展示。

2.5.2 任务条

每个任务在对应日期区间显示横条。条的颜色可区分普通任务与里程碑；完成度以进度条形式叠加显示。status_date 用于计算基于时间的完成度。

2.5.3 左侧表格

左侧固定列显示：序号、选择框、任务名称、责任人、工期、开始日期、前置任务、依赖类型等。支持列宽调整与排序。

2.5.4 依赖线

任务间依赖以箭头线连接。从前置任务结束指向后续任务开始。支持创建、删除、高亮选中。

2.5.5 搜索与筛选

支持按任务名称搜索，匹配项高亮并可聚焦。支持全部展开/折叠层级。

2.6 依赖管理功能

2.6.1 创建依赖

在甘特图中从一任务条拖线到另一任务条，或通过表单选择 from_task、to_task 创建依赖。默认类型为 FS（Finish-to-Start），可设置 lag。

2.6.2 删除依赖

点击依赖线或从列表中删除。删除后甘特图不再显示该连线。

2.6.3 依赖类型

支持 FS（完成-开始）、SS（开始-开始）、FF（完成-完成）、SF（开始-完成）等类型，type 字段存储对应枚举值。

2.7 版本历史功能

2.7.1 创建版本

用户点击「保存版本」时，系统将当前任务与依赖的完整状态序列化为 JSON 存入 project_versions，并生成 change_logs 记录相对上一版本的变更。

2.7.2 版本列表

以时间倒序展示版本号、创建时间、变更数量。支持查看某版本的快照与变更明细。

2.7.3 版本对比

可对比两个版本的差异，显示新增、修改、删除的任务及依赖变化（功能可扩展）。

2.8 任务生命周期审计

2.8.1 事件类型

每次任务创建、更新、移动、删除均写入 task_lifecycle。event_type 为 created/updated/moved/deleted；field_name 与 old_value、new_value 记录具体变更。

2.8.2 历史查看

在任务编辑弹窗的「历史」标签中，按时间倒序展示该任务的所有 lifecycle 事件，便于审计与问题追溯。

第三部分 用户手册

3.1 系统安装与启动

3.1.1 环境准备

确保已安装 Node.js 20 及以上、PostgreSQL 14 及以上。从项目根目录执行 npm install 安装依赖。

3.1.2 数据库配置

在 lib/db.ts 中配置数据库连接（host、port、database、user、password）。执行 npm run migrate 或 node scripts/migrate.ts 初始化数据库表结构。

3.1.3 启动服务

执行 npm run dev 启动开发服务器，默认访问 http://localhost:3000。生产环境使用 npm run build 构建后，以 npm run start 启动。

3.2 注册与登录

3.2.1 注册新用户

打开系统后，若未登录将跳转到登录页。点击「注册」进入注册页面，填写用户名、邮箱、密码，提交后完成注册并自动登录。

3.2.2 登录

在登录页输入用户名和密码，点击「登录」。登录成功后进入仪表盘，显示「你好，[用户名]」。

3.2.3 退出登录

点击右上角「退出登录」即可退出，返回登录页。

3.3 项目管理操作

3.3.1 查看项目列表

登录后进入仪表盘，显示「我的项目」列表。每个项目以卡片形式展示，包含项目名称等信息。

3.3.2 新建项目

点击「+ 新建项目」，输入项目名称，点击「创建」。创建成功后项目出现在列表中，可点击进入甘特图。

3.3.3 进入项目

点击项目卡片进入该项目甘特图页面，顶部显示项目名称和「返回项目列表」链接。

3.4 甘特图操作

3.4.1 界面布局

甘特图页面分为：顶部工具栏、左侧任务表格、右侧时间轴与任务条区域。工具栏提供缩放、搜索、展开/折叠、版本等按钮。

3.4.2 缩放

点击「放大」或「缩小」调整时间轴列宽，从而改变时间粒度（更细或更粗）。

3.4.3 创建任务

通过工具栏或右键菜单创建任务。输入任务名称后，可设置开始日期、结束日期、责任人、完成度等。子任务可指定父任务创建。

3.4.4 编辑任务

双击任务行或点击编辑按钮打开编辑弹窗。在「编辑」标签中修改任务属性，点击「保存」完成更新。在「历史」标签可查看该任务的变更记录。

3.4.5 删除任务

选中任务后点击删除或按 Delete 键，确认后任务被软删除，不再显示在甘特图中。

3.4.6 拖拽调整时间

鼠标悬停任务条左/右边缘，光标变为左右箭头时，可拖拽调整任务的开始或结束时间，工期随之变化。

3.4.7 拖拽调整顺序

拖拽任务行左侧的拖拽手柄，可调整任务在列表中的顺序，影响 order_index。

3.4.8 创建依赖

从一任务条的右侧拖出连线到另一任务条，即可创建依赖关系。依赖以箭头线表示，从前置任务指向后续任务。

3.4.9 删除依赖

点击依赖线选中后，按 Delete 或点击删除按钮可移除该依赖。

3.4.10 搜索任务

在工具栏搜索框输入关键词，匹配的任务名称高亮显示，可快速定位任务。

3.4.11 展开与折叠

点击「全部展开」或「全部折叠」可展开或折叠所有有子任务的任务行。

3.5 版本管理操作

3.5.1 保存版本

点击工具栏「版本」或「保存版本」，系统将当前任务与依赖状态保存为新版本，并记录变更日志。

3.5.2 查看版本列表

点击「版本」打开版本面板，显示所有版本号、创建时间、变更数量。点击某版本可查看详情。

3.5.3 查看版本详情

在版本详情中可查看该版本快照的完整任务与依赖数据，以及相对上一版本的变更列表（新增、修改、删除的任务及字段）。

3.6 常见问题

3.6.1 无法登录

检查用户名和密码是否正确；确认数据库连接正常；查看浏览器控制台是否有网络错误。

3.6.2 任务不显示

确认任务未被软删除；检查项目是否加载完成；尝试刷新页面。

3.6.3 依赖线不显示

确认两个任务均存在且未删除；检查依赖是否已正确保存到数据库。

3.6.4 版本保存失败

确认有任务变更；检查网络与 API 权限；查看服务端日志。

第四部分 系统流程图

4.1 用户注册流程

开始 → 用户填写用户名、邮箱、密码 → 前端校验格式 → 后端校验唯一性 → 密码 bcrypt 加密 → 插入 users 表 → 签发 JWT → 返回用户信息与 token → 前端存储 token → 跳转仪表盘 → 结束。

4.2 用户登录流程

开始 → 用户输入用户名、密码 → 后端根据用户名查询 users → 校验 bcrypt 密码 → 签发 JWT → 返回 token → 前端存储 → 跳转仪表盘 → 结束。若校验失败，返回错误信息。

4.3 项目创建流程

开始 → 用户点击新建项目 → 输入项目名称 → 提交 POST /api/projects → 校验 JWT → 插入 projects 表（user_id 为当前用户）→ 返回项目信息 → 前端加入列表 → 结束。

4.4 任务创建流程

开始 → 用户点击创建任务 → 填写名称等属性 → 提交 POST /api/tasks/[projectId] → 校验项目归属 → 生成 task_code → 插入 tasks 表 → 可选写入 task_lifecycle（created）→ 返回任务 → 前端加入 Redux 并重绘甘特图 → 结束。

4.5 任务更新流程

开始 → 用户编辑任务并保存 → 提交 PUT /api/tasks/[projectId]（含任务数组）→ 校验归属 → 更新 tasks 表 → 写入 task_lifecycle（updated，含字段变更）→ 返回 → 前端更新 Redux → 重绘 → 结束。

4.6 依赖创建流程

开始 → 用户从任务 A 拖线到任务 B → 提交 POST /api/dependencies/[projectId] → 校验 from_task、to_task 存在且无循环 → 插入 dependencies 表 → 返回 → 前端添加依赖并绘制连线 → 结束。

4.7 版本保存流程

开始 → 用户点击保存版本 → 前端可选先 dispatch saveSnapshot 同步本地状态 → 提交 POST /api/versions/[projectId] → 后端获取当前任务与依赖 → 序列化为 JSON → 插入 project_versions → 计算与上一版本的 diff → 写入 change_logs → 返回版本信息 → 前端刷新版本列表 → 结束。

4.8 甘特图渲染流程

开始 → 加载任务与依赖（GET /api/tasks、/api/dependencies）→ 存入 Redux → 计算扁平化任务树（含层级、展开状态）→ 计算时间轴范围 → 按行渲染左侧表格 → 按任务条渲染右侧时间条 → 绘制依赖箭头线 → 处理拖拽、点击等交互 → 结束。

第五部分 开发情况

5.1 开发周期

项目开发周期约 4 周，分为需求分析、设计、编码、测试四个阶段。需求分析阶段确定功能范围与优先级；设计阶段完成数据库设计、接口设计、模块划分；编码阶段按模块迭代实现；测试阶段进行功能测试与回归测试。

5.2 开发环境

开发使用 Windows/macOS，IDE 为 VS Code。版本控制为 Git。数据库使用 PostgreSQL 14 本地安装。Node.js 版本 20 LTS。

5.3 代码统计

截至文档编写时，项目自有代码（排除 node_modules、.next、public 第三方库）约 4956 行，其中 TypeScript/TSX 约 4931 行，CSS 约 17 行，配置文件约 8 行。主要模块行数：GanttChart 组件约 1631 行，GanttToolbar 约 470 行，任务 API 约 353 行，版本 API 约 368 行。

5.4 技术难点与解决

5.4.1 甘特图拖拽与依赖线

任务条拖拽需考虑时间轴坐标与日期的转换，以及依赖约束下的合法区间。依赖线绘制需计算任务条的位置，使用 SVG 或 Canvas 绘制贝塞尔曲线或折线。采用 React state 管理拖拽状态，在 mouseup 时提交 API。

5.4.2 版本快照与变更计算

版本快照需序列化当前任务与依赖的完整状态。变更计算通过对比上一版本快照与当前状态，生成 change_logs。使用深度比较算法识别新增、修改、删除的任务及字段级变更。

5.4.3 任务层级与展开折叠

任务按 parent_id 构建树形结构，再扁平化为渲染用的行列表，每行记录 level 与 expanded 状态。展开/折叠时更新 expanded 状态，过滤或包含子任务行。

5.4.4 基于时间的完成度

当项目设置 status_date 时，任务完成度可按时间线性插值：若 status_date 早于开始日期则为 0，晚于结束日期则为 100，否则按 (status_date - start) / (end - start) 计算。用于「当前日期」视角的进度展示。

5.5 后续优化计划

计划增加：导出甘特图为图片或 PDF；任务模板与批量导入；更多依赖类型与 lag 的 UI 配置；多用户协作与权限细分；移动端适配；性能优化（虚拟滚动、按需加载）。

第六部分 测试结果

6.1 测试环境

测试环境与开发环境一致：Node.js 20，PostgreSQL 14，Chrome 最新版。使用手动功能测试为主，辅以 API 的简单请求验证。

6.2 功能测试

6.2.1 认证测试

注册：输入新用户名、邮箱、密码，提交后成功创建用户并跳转仪表盘。重复用户名或邮箱注册，返回错误提示。
登录：正确用户名密码可登录；错误密码返回错误。
登出：点击登出后返回登录页，再次访问需认证接口返回 401。

6.2.2 项目测试

创建项目：输入名称后成功创建，列表中出现新项目。

进入项目：点击项目卡片进入甘特图，顶部显示项目名称。
删除项目：删除后列表移除，其下任务与依赖在数据库中已级联删除。

6.2.3 任务测试

创建任务：可创建根任务与子任务，task_code 自动递增。
编辑任务：修改名称、日期、责任人、完成度等，保存后正确更新。
删除任务：软删除后甘特图不再显示，依赖关系过滤正确。
拖拽：拖拽任务条调整时间，工期与日期联动正确；拖拽行调整顺序，order_index 更新正确。

6.2.4 依赖测试

创建依赖：从任务 A 拖线到任务 B，依赖线正确显示，数据库中有记录。
删除依赖：删除后连线消失，数据库记录移除。
循环检测：若存在循环依赖，创建时后端应拒绝（根据实现）。

6.2.5 版本测试

保存版本：点击保存版本后，版本列表增加新条目，版本号递增。
查看版本：可查看版本快照与变更日志，数据与预期一致。

6.2.6 任务历史测试

编辑任务并保存后，在「历史」标签中可见 updated 类型事件，字段变更正确记录。

6.3 兼容性测试

在 Chrome、Edge、Firefox 最新版中，主要功能（登录、项目与任务 CRUD、甘特图拖拽、依赖线）均正常。Safari 未做完整测试。

6.4 性能测试

在任务数量约 100、依赖约 50 条时，甘特图渲染与拖拽流畅。超过 500 任务时，可考虑虚拟滚动等优化。API 响应时间在本地环境下均小于 200ms。

6.5 已知问题

（1）部分浏览器下依赖线在缩放时可能轻微错位，待优化坐标计算。（2）版本对比功能尚未实现，仅支持查看单版本快照与变更日志。（3）移动端触摸操作支持有限，建议使用桌面浏览器。

6.6 测试结论

核心功能（用户管理、项目管理、任务管理、甘特图展示与交互、依赖管理、版本历史、任务审计）均通过功能测试，满足设计要求。系统可进入交付与部署阶段，后续根据用户反馈持续改进。

附录 A 术语表

甘特图：以横条形式在时间轴上展示任务及其进度的图表。
依赖：任务间的先后关系，如 A 完成后 B 才能开始。
FS：Finish-to-Start，完成-开始型依赖。
里程碑：工期为零、表示关键节点的任务。
软删除：逻辑删除，数据仍保留但标记为已删除，不再展示。
JWT：JSON Web Token，用于无状态认证。
Redux：JavaScript 状态管理库。
API：Application Programming Interface，应用程序接口。

附录 B 参考文档

- Next.js 官方文档：https://nextjs.org/docs

- React 官方文档：https://react.dev

- Redux Toolkit：https://redux-toolkit.js.org
- PostgreSQL 文档：https://www.postgresql.org/docs
- Tailwind CSS：https://tailwindcss.com

附录 C 详细设计补充

C.1 甘特图组件详细设计

GanttChart 组件是系统的核心展示组件，负责将任务数据可视化为时间轴上的横条。组件接收 projectId、statusDate、colW、searchQuery 等 props。内部维护 expanded 状态控制任务树的展开折叠，editId 和 editName 控制行内编辑，drag 状态管理任务条拖拽，connect 状态管理依赖线绘制。时间轴采用双层表头：上层显示月份，下层显示日期。任务条通过计算 start_date 和 end_date 在时间轴上的像素偏移来定位，条高固定为 BAR_H（20px），垂直居中于行内。依赖线使用 SVG 绘制，从源任务条右端到目标任务条左端，采用折线或贝塞尔曲线避免重叠。

C.2 任务状态管理详细设计

任务数据存储在 Redux 的 tasksSlice 中，包含 tasks 数组和 dependencies 数组。selectedIds 记录当前选中的任务，clipboard 用于复制粘贴。actions 包括 setTasks、addTasks、updateTasks、deleteTasks、addDependency、removeDependency、updateDependency、saveSnapshot、copyTasks 等。saveSnapshot 将当前状态序列化后存入内存，用于版本保存前的数据准备。任务更新采用乐观更新策略：先更新 Redux，再调用 API，失败时回滚。

C.3 API 中间件与鉴权

所有需认证的 API 通过 getAuthUser(req) 获取当前用户。该函数从 Cookie 或 Authorization 头中解析 JWT，验证签名与有效期，返回 userId 和 username。若 token 无效或缺失，返回 failure 对象，API 响应 401。项目与任务相关 API 还需 verifyProjectOwnership 确保用户只能操作自己的项目。中间件在 lib/middleware.ts 中配置，对 /api 路径进行统一处理。

C.4 数据库连接池

使用 pg 库的 Pool 管理数据库连接。在开发环境中，通过 globalThis 缓存 pool 实例，避免 Next.js 热重载时重复创建连接。连接配置从 lib/db.ts 读取，生产环境建议通过环境变量注入 host、port、database、user、password，避免硬编码。

C.5 错误处理与结果封装

API 统一使用 lib/result.ts 的 success 和 failure 函数封装响应。success(data) 返回 { ok: true, value: data }；failure(message, code) 返回 { ok: false, error: message, code }。前端根据 ok 字段判断请求是否成功，失败时展示 error 信息。

C.6 前端路由与权限

未登录用户访问 /dashboard 或 /projects/[id] 时，通过 useEffect 检测 user 状态，若为空则 router.push('/login')。登录后 token 存储在 cookie 中，页面加载时通过 /api/auth/me 或类似接口恢复用户信息到 Redux。登出时清除 cookie 并 dispatch logout()，清空 auth 状态。

C.7 任务编码生成规则

task_code 格式为 T-XXX，其中 XXX 为三位数字，从 001 递增。生成逻辑：查询该项目下现有任务中 task_code 的最大数字部分，加 1 后格式化为三位字符串。新项目首个任务为 T-001，依次递增。编码在任务创建时由后端生成，前端只读展示。

C.8 依赖类型与 lag 说明

type 字段：1=SS（开始-开始），2=FS（完成-开始），3=FF（完成-完成），4=SF（开始-完成）。lag 为滞后量，单位为天，正数表示延迟，负数表示提前。例如 FS 依赖 lag=2 表示前置任务完成后 2 天，后续任务才能开始。当前 UI 主要支持 FS，其他类型可扩展。

C.9 版本快照结构

snapshot 为 JSONB，结构为 { tasks: Task[], dependencies: Dependency[] }。保存版本时，从数据库查询当前项目下未删除的任务与依赖，序列化后写入。读取版本时解析 JSON，可渲染为只读甘特图或用于对比。change_logs 记录相对上一版本的变更：change_type 为 task_created、task_updated、task_deleted、dependency_created、dependency_deleted 等；task_name、field_name、old_value、new_value 提供变更明细。

C.10 软删除与级联

任务软删除时设置 is_deleted=true、deleted_at=NOW()。查询任务时默认加上 AND is_deleted=false 条件。依赖查询时 JOIN tasks 并过滤 is_deleted，确保不显示涉及已删除任务的依赖线。项目删除为硬删除，使用 ON DELETE CASCADE 级联删除其下所有任务、依赖、版本、变更日志。

附录 D 界面说明

D.1 登录页

居中表单，包含用户名输入框、密码输入框、登录按钮、注册链接。样式简洁，采用 Tailwind 的 border、rounded、shadow 等类。错误信息以红色文字显示在表单下方。

D.2 注册页

与登录页类似，增加邮箱输入框。提交后若成功，自动登录并跳转；若失败，显示具体错误（如用户名已存在）。

D.3 仪表盘

顶部导航栏：左侧显示「甘特图管理」标题，右侧显示「你好，[用户名]」和「退出登录」按钮。主区域为项目卡片网格，每卡片显示项目名称，点击进入项目。右上角「+ 新建项目」按钮，点击后展开输入框，输入名称后创建。

D.4 甘特图页

顶部：返回链接、项目名称。工具栏：放大、缩小、搜索框、全部展开、全部折叠、聚焦任务、版本按钮。主体：左侧固定列表（序号、复选框、任务名、责任人、工期、开始日期、前置、类型），右侧横向滚动的时间轴与任务条。依赖线叠加在任务条上方。底部可预留状态栏显示选中任务数等。

D.5 任务编辑弹窗

模态框，包含「编辑」和「历史」两个标签。编辑标签：任务名称、责任人、开始日期、结束日期、完成度滑块、里程碑复选框、备注多行文本。保存和取消按钮。历史标签：按时间倒序列出 lifecycle 事件，每项显示类型、字段、旧值、新值、描述、时间。

D.6 版本面板

侧边抽屉或模态框，列表显示版本号、创建时间、变更数量。点击某版本展开详情，显示快照摘要或变更日志列表。可支持「对比」按钮（待实现）。

文档结束